

Object-Oriented Programming (Classes and Objects)

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor(CSE), PIT
Parul University

1. Parameter Passing.....	1
2. Recursion with Method Calls	2
3. Creating and Using Nested Classes	3
4. Summary of Key Concepts.....	4
5. References.....	4

5.4.1 Parameter Passing

Java uses **pass-by-value** for all variable passing:

- For **primitive types**, the actual value is passed.
- For **reference types**, the reference (i.e., memory address) is passed **by value** – so object contents can be changed, but not the reference itself.

◆ Call by Value Mechanism in Java

Java strictly uses Call by Value for all method calls, regardless of whether the parameter is a primitive type or a reference type.

✓ Definition

In Call by Value, a copy of the actual value is passed to the method. Changes made to the parameter inside the method do not affect the original variable.

◆ Call by Value with Primitives

When you pass a primitive (e.g., int, float), Java copies its value.

```
void changeValue(int a) {  
    a = a + 5;  
}  
  
public static void main(String[] args) {  
    int x = 10;  
    changeValue(x);  
    System.out.println(x); // Output: 10 (no change)  
}
```

Note : x is not changed because a copy of x was modified.

◆ Call by Value with Object References

Java passes the reference by value, meaning a copy of the reference is passed, not the actual object.

```
void modify(Student s) {  
    s.name = "Updated"; // modifies the original object  
}  
  
public static void main(String[] args) {  
    Student obj = new Student("Original");  
    modify(obj);  
    System.out.println(obj.name); // Output: Updated  
}
```

- Object content is modified because both references point to the same object.
- But reassigning the object reference inside the method does not affect the original reference.
- **Example which does NOT affect original reference**

```
void modify(Student s) {
    s = new Student("New"); // does NOT affect original reference
}
```

5.4.2 Recursion with Method Calls

✓ What is Recursion?

Recursion is a programming technique where a method **calls itself** to solve a problem in smaller sub-problems.

◆ Syntax and Structure

A recursive method must have:

1. **Base Case** – Condition to stop recursion.
2. **Recursive Call** – The method calls itself with a changed argument.

```
int factorial(int n) {
    if (n == 0) return 1; // base case
    else return n * factorial(n - 1); // recursive call
}
```

◆ Use Cases

- Factorial calculation
- Fibonacci series
- Tree traversal
- Searching and sorting (e.g., QuickSort, MergeSort)

◆ Pros and Cons

Pros	Cons
Simpler code for complex problems	Can cause StackOverflowError
Easier to understand divide-and-conquer algorithms	More memory usage (stack frames)

5.4.3 Creating and Using Nested Classes

✓ What is a Nested Class?

A **nested class** is a class declared inside another class. It helps in **grouping related logic** and **encapsulating helper classes**.

◆ Types of Nested Classes

1. Static Nested Class

- Doesn't require an instance of the outer class.
- Can only access static members of the outer class.

```
class Outer {  
    static class StaticNested {  
        void show() {  
            System.out.println("Static nested class");  
        }  
    }  
}
```

2. Non-static Inner Class

- Requires an instance of the outer class.
- Can access all (including private) members of the outer class.

```
class Outer {  
    class Inner {  
        void display() {  
            System.out.println("Inner class");  
        }  
    }  
}
```

3. Local Inner Class

- Defined inside a method.
- Scoped only to that method.

```
void method() {  
    class LocalInner {  
        void msg() {  
            System.out.println("Local inner class");  
        }  
    }  
}
```

4. Anonymous Inner Class

- Defined and instantiated in one step.
- Used for implementing interfaces or overriding methods without formally declaring a class.

```
Runnable r = new Runnable() {  
    public void run() {  
        System.out.println("Anonymous inner class");  
    }  
};
```

◆ Advantages of Nested Classes

- Increases encapsulation.
- Logical grouping of classes that are only used in one place.
- Enhances code readability and maintainability.

➤ Summary of Key Concepts:

- **Parameter Passing (Call by Value):**
 - Java uses call by value only, meaning a copy of the variable is passed to methods.
 - For objects, the reference is passed by value, so changes to object fields reflect outside the method.
- **Recursion:**
 - A method that calls itself for solving problems in smaller steps.
 - Requires a base case and recursive call.
 - Commonly used in algorithms like factorial, Fibonacci, and tree traversal.
- **Nested Classes:**
 - Classes defined within another class for better encapsulation and organization.
 - **Types:** Static Nested, Non-static Inner, Local Inner, and Anonymous Inner.
 - Improves code readability and modularity.

➤ References:

- Oracle Official Documentation:
 - [Nested Classes](#)
 - [Passing Information to a Method](#)

GeeksforGeeks

1. ◆ **Call by Value in Java**
<https://www.geeksforgeeks.org/java-is-pass-by-value/>
2. ◆ **Recursion in Java**
<https://www.geeksforgeeks.org/recursion-in-java/>
3. ◆ **Nested Classes in Java**

<https://www.geeksforgeeks.org/nested-classes-java/>

 TutorialsPoint

1. **Java – Methods**

https://www.tutorialspoint.com/java/java_methods.htm

2. **Java – Inner/Nested Classes**

https://www.tutorialspoint.com/java/java_innerclasses.htm

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | **GRADE**



    
<https://paruluniversity.ac.in/>